Chess^2

Team 10

Version <1.1>

Team Members	Roles	Github
Carson Schraad	Co-Scrum Master	CarsonSchraad
Collin Tullis	UI/UX Designer	monrester
Ethan Cook	Quality Assurance	EthansCookin
Ethan Le	Configuration (Version Control) Manager & Project Log Scribe	27EthanL
Kristoffer Roy Comahig	Technical Writer & Quality Assurance	KrisC341
Maha Jornaz	Project Manager	Maha-Jor
Phoenix Brehm	Technical Lead	phoenixbrehm

Project Synopsis:

An implementation of chess, with a pre-game “shop” for selecting which pieces the player will have at their disposal with a randomized point budget.

Architecture:

Purpose:

This document outlines the architecture of Chess^2, a multiplayer game that is built upon the classic game of Chess with additional game mechanics such as a randomized point budget used by players to ‘buy’ what pieces they will have at their disposal during a game, new original chess pieces, and various powerups that alter the traditional gameplay loop of Chess.

The core gameplay loop and subsequent data flow of Chess^2 can be described as follows.

When a singular game starts, both players receive a randomized point budget. With these point budgets, each player is able to ‘buy’ or select what pieces they would like to start out the game with to use such as queens, pawns, rooks, etc. The only requirement is that they have a single King which always starts at squares e1 and e8 for the white and black king respectively. Regardless of whatever pieces are chosen or what size the nxn square board currently is, the objective of every game is to checkmate the enemy king or to cause a draw.

Once each player has bought their pieces and selected where they would like those pieces placed, the proper repeated gameplay loop or data flow of Chess^2 starts. White completes a turn first with subsequent iterations in the loop having the players alternate turns. One piece is moved by a player per turn as in traditional chess. Pieces can capture enemy pieces by moving to an enemy-occupied square, taking its place on the board. Once a move has been completed, the user interface calls upon the board layer to check if any pieces have been captured or if a check or checkmate has been done. Depending on the result, the board is updated and players are promptly notified of any major game states such as check or checkmate. The gameplay loop then repeats until one player is able to get a checkmate against their opponent.

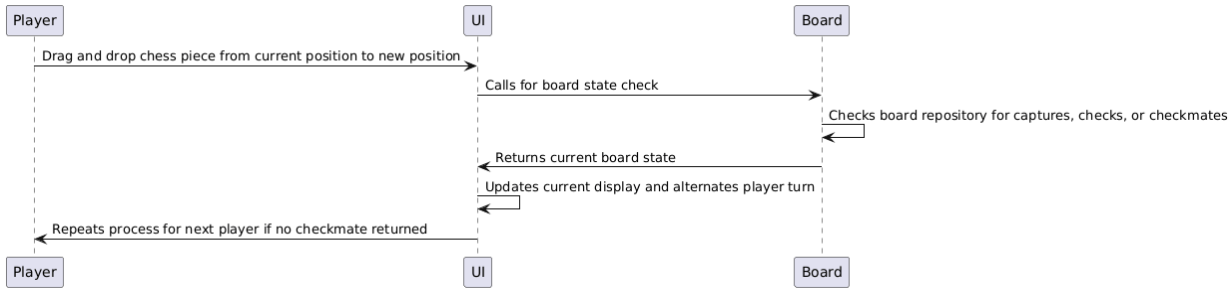
Overview:

The software architecture of Chess^2 utilizes a mixed architecture, combining layered and repository-based architecture. Chess^2 has two layers: the user interface and the chess board.

The user interface layer takes input from the user regarding what pieces they would like to buy before a round starts, what piece they can and would like to move along with where, etc. It also handles a majority of enforcement when it comes to the data / gameplay flow of chess, In addition to taking input, the user interface displays the current state of the game: primarily what pieces are on the board, where those

pieces are, and whose turn it is amongst the players, It sends input to the chess board layer below it and outputs data sent to it by the chess board layer.

The chess board layer is based on a repository architecture style. The input received from the user interface is filtered down into the ‘board’ module which effectively acts as a central data store of what pieces are currently active and where. These pieces are considered independent components, being individual objects within the program. Each piece object has its own rules for movement, point value, and data storage for possible powerups or mechanisms that it consults when interacting with the board during gameplay turns. The input from the user interface combined with the data held by each individual piece all serve to facilitate a classic game of Chess and its various states on the board central data store.

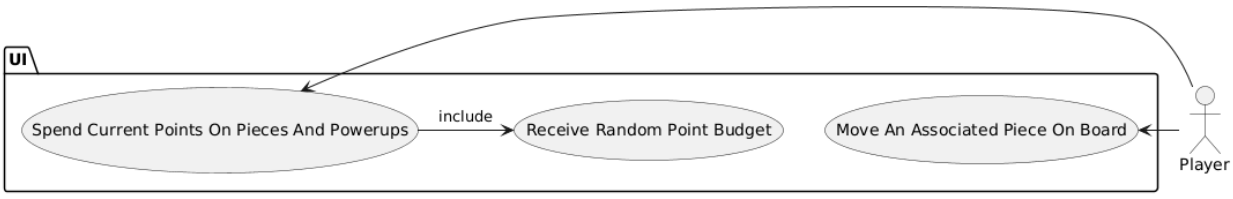


User Interface Layer:

This layer will be a graphical user interface that utilizes clicks and drags to take user input regarding commands such as buying pieces during initial game startup or where they would like to move a given piece on the board. The User Interface layer will be a prime enforcer of the traditional gameplay loop, restricting what pieces are able to be moved in a turn dependent on which player’s turn it even is.

While the board layer’s current state within its central data store is ultimately hidden from the user, the User Interface Layer will provide a filtered view of said state via the chess board.

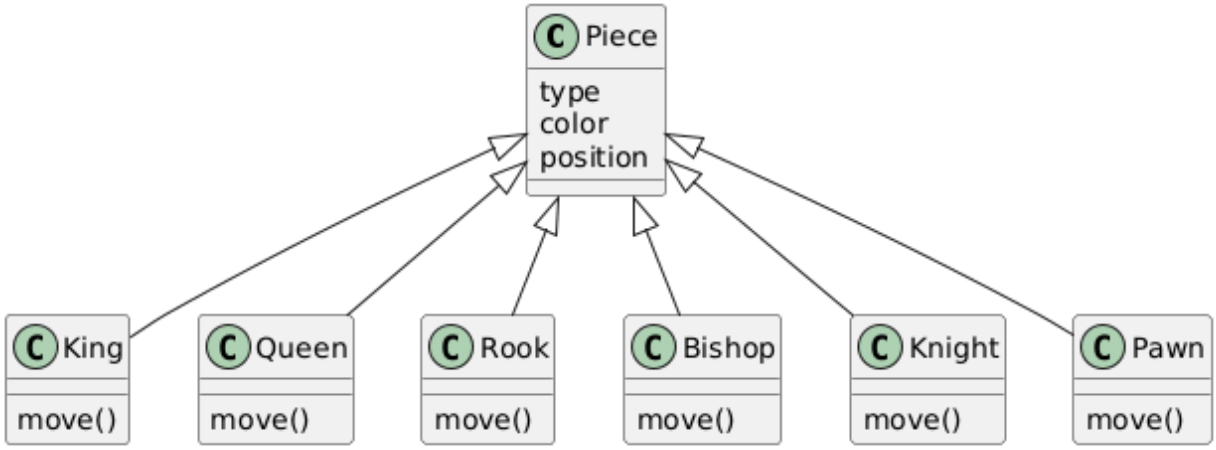
This User Interface layer will utilize sprites and images representative of chess pieces such as the Knight or the Rook or interactive elements such as buttons.



Board Layer / Repository:

The Chess Board Layer or Repository stores the current state of the chess game within its central repository of board. This current state consists of what pieces are currently on the board, where those pieces are, and what roguelike mechanics may be active for pieces or the game overall.

The central repository ultimately stores pieces in an array representative of a traditional chess board. These pieces are objects that store movement rules, point value, and data storage for possible powerups or mechanisms that it consults when interacting with the board during gameplay turns. Each piece is fully aware of the board state stored in the central repository, even if it doesn't utilize it, in order to ensure each piece follows the rules of Chess and active roguelike elements regardless of the active combination of pieces and mechanics. The central repository also stores Boolean values that indicate active roguelike elements, which can alter attributes within each piece to change the flow of rules and gameplay.



[*above figure is outdated and will need updating once all pieces implemented*]

Piece Superclass:

The Piece superclass is the class from which specific pieces such as the Pawn and the King derive their framework from through inheritance. Piece has seven attributes: color, position, board, pointVal, directions, attackDir, and sliding.

While the methods for Piece are listed below, they may be overwritten depending on a piece's specific needs.

getColor():

This method returns the color of the piece for UI purposes.

getPos():

This method returns the current position of the piece on the board in tuple format.

get_valid_moves():

This method iterates through all cells on the board that are possible directions for the piece to move and determines which piece is in what cell. If there is no piece, it is a valid move. If there is a piece but it is opposing, it additionally is a valid move. Any valid moves are then returned, provided they do not leave king in check.

move():

This method moves the piece to a new position if it is a valid move and updates the board in the process by removing the piece from its old position to its new inputted position.

get_attack_squares():

This method is similar in execution to that of get_valid_moves() but focuses on determining if a piece is in the path of movement that will be attacked.

does_move_leave_king_in_check():

This method is a movement method that updates the pieces move along with calling board's is_color_in_check() method to determine if a specific move leaves a king in check.

Pawn Subclass:

In terms of differences in attributes to the Piece Superclass and all other subclasses, Pawn's direction comes with the built-in understanding that it may need to be multiplied by -1 in the scenario of black movement, though this multiplication is not built into the class attribute itself.

get_valid_moves():

This method is fairly similar to that of its superclass but with three major differences. First, it removes detecting if the piece found on the cell its iterating through is the same color as the pawn. Second, it accounts for double movement of the pawn on its first turn and tests if the current iterated cell is a valid double move. Three, it detects possible attack squares and adds them in as valid moves if there are pieces there.

move():

This method adds some additional functionality related to the Pawn to Piece's move(). After Piece's move() is called, if the move wasn't valid, it will still be returned as invalid. Additionally,

the Pawn’s first move is now marked as taken and if any conditions for promotion are met, the promotion() method can be called.

promotion():

Depending on the user’s choice of bishop, knight, rook, or queen, a new piece of said class will be initialized and returned. This is meant to replace the pawn.